The `surfacePosition` function object: implementation and tests

Johan Roenby

September 17, 2026

Abstract

`surfacePosition` is an OpenFOAM function object that reports where a volume-of-fluid interface crosses user-specified lines. It is an alternative to OpenFOAM's `interfaceHeight`, which integrates the volume fraction along a sampled line. `surfacePosition` finds the cells crossed by each line once and follows them through mesh changes by a flood fill, so its cost per write does not depend on the mesh size, and it computes the crossing geometrically from the interface plane in the cell. This report describes the implementation and the test suite in `test`. All numbers, tables and figures are generated from the output of one run of the suite (wall time 1 min 40 s). In a 1 000 000-cell case on 18 processors, `surfacePosition` costs 2.4 ms per time step for 50 lines, against 1278 ms for `interfaceHeight`, and on the exact stream function wave it is more accurate than `interfaceHeight` on the same mesh.

1 Usage

`surfacePosition` is built into the library `libsurfacePosition` by `Allwmake` in its repository. It links only OpenFOAM's `finiteVolume`, `meshTools`, `sampling` and `lagrangian` libraries. Lines are given either as locations and a direction, as for `interfaceHeight`, or as explicit start and end points:

```
surfacePosition1
{
    type            surfacePosition;
    libs            (surfacePosition);
    locations       ((1.0001 0.0001 0.0001));
    // or explicit lines: lines (((x0 y0 z0) (x1 y1 z1)));
    direction       (0 0 1);        // optional, default -g
    alpha           alpha.water;    // optional
    method          auto;           // reconstructed, plic, alpha or auto
    alphaTol        1e-8;           // optional
    checkLineCells  false;          // tests: compare line cells with all cells
    fullSearchInterval 0;           // full search every N mesh changes
    writeControl    timeStep;
    writeInterval   1;
}
```

The file `postProcessing/<name>/<time>/surfacePosition.dat` holds, per line, the distance s from the line start, the height h above the location (locations only) and the point (x, y, z). A line without an interface is written as `nan`.

2 Implementation

2.1 Lines

With locations, each line runs through the location along $-\mathbf{d}$, where $\mathbf{d}$ is the upward direction, from above the mesh to below it, across the bounding box of the mesh plus a margin of the box

diagonal for mesh motion. This holds wherever the location is, also outside the mesh, and the first interface found from the start is the uppermost one. Explicit lines are used as given; the first interface from their start is reported.

2.2 Cells crossed by a line

Whether a line $\mathbf{x}(s) = \mathbf{x}_0 + s\,\mathbf{t}$, $0 \le s \le L$, crosses a convex cell is decided by clipping the parameter interval against the half-spaces of the cell faces. For a face with outward area vector $\mathbf{S}_f$ and centre $\mathbf{x}_f$, let $a = \mathbf{S}_f \cdot \mathbf{t}$ and $b = \mathbf{S}_f \cdot (\mathbf{x}_f - \mathbf{x}_0)$. For $a > 0$ the interval is cut above at $s = b/a$ (the line leaves), for $a < 0$ below (the line enters, and the face is recorded as the entry face). The face is taken as parallel to the line if the distance of the line from the face plane changes by less than a tolerance along the whole line,

$$|a|\,L \le \tau, \qquad \tau = 10^{-9}\,|\mathbf{S}_f|^{3/2}, \tag{1}$$

and the cell is then rejected only if $b < -\tau$. A line lying in a face plane is thereby inside the cells on both sides. The cell is crossed if the clipped interval has a length above $10^{-15}L$; its ends s_{in} and s_{out} are stored with the cells, which are sorted along the line.

The cells are found once by a `faceOnlySet` with a cached `meshSearch`, plus the cells nearest to five points of the line inside the local mesh bounds, which catch lines lying on faces. These cells seed the flood fill below, which completes the set.

Flood fill. After a mesh change the cells are found again without a search tree, by a flood fill that tests the face neighbours of cells crossing the line. The seeds are the old line cells, mapped through `reverseCellMap` after a topology change (`updateMesh`) and unchanged after mesh motion (`movePoints`). If no seed crosses the line any more, rings of face neighbours around the seeds are tested, up to 10 rings, until a ring holds a cell crossing the line.

The flood fill and the ring search continue across processor faces: a candidate cell reaching a processor face sends the cell on the other side to the neighbouring processor, with the number of rings still to search around it. All lines are processed together, with one exchange per round, until no processor has candidates to send. A line that moves into a neighbouring processor subdomain is thereby followed (section 3.3). A line that moves further than the ring limit, or that is connected to its old cells only through cyclic faces, is only found by a full search, done every `fullSearchInterval` mesh changes if requested.

With `checkLineCells`, the cells found after every mesh change are compared with all cells of the mesh that cross the line, and the numbers of missing and extra cells are written to the log. The test suite uses this to verify the flood fill.

2.3 Interface position in a cell

Three methods are available:

reconstructed The interface plane of the geometric VoF reconstruction, for solvers such as `interIsoFoam`. The normal and centre are read from the fields `interfaceNormal.<phase>` and `interfaceCentre.<phase>`, which the reconstruction schemes register on the mesh and update at the end of the advection step. `surfacePosition` depends only on these names, not on the reconstruction classes, so it works with any library that registers them (such as OpenFOAM's `geometricVoF` or a modified copy) without linking to it. Interface cells are cells with a non-zero normal.

plic A plane with the Youngs normal (the Gauss gradient of α interpolated to the cell points) positioned by bisection so that it cuts the volume fraction α from the cell. The cut volume is summed over tetrahedra from the cell centre, face centres and face edges. Interface cells have $\alpha_{\text{tol}} < \alpha < 1 - \alpha_{\text{tol}}$. This method works with any solver; the point interpolation of α covers the whole mesh once per write.

alpha The point where $\alpha = 0.5$, interpolated linearly between the centres of each line cell and the cell before it along the line, also across processor and cyclic faces.

The default, `auto`, uses `reconstructed` if the fields are registered and `plic` otherwise.

2.4 First interface along a line

The cells of a line are visited in order. The crossing of a cell's interface plane with the line is an *event* if it lies within $[s_{\text{in}}, s_{\text{out}}]$ of the cell (to a relative tolerance of 10^{-6}), and the smallest event distance is used. A sharp interface on a face between two full cells (α within 10^{-6} of 0 and 1 on either side of the entry face, also across processor and cyclic faces) is an event at s_{in}. Only if a line has no event anywhere is the first plane crossing outside its cell used as a fallback. The visit stops once s_{in} exceeds the best event.

2.5 Parallel runs

Each processor finds the first event and fallback of each line on its part of the line, and one `listReduce` with a minimum over all lines selects the first along the whole line. Only the distance is reduced, so a point found by several cells or processors (an interface or a line on a face, edge or point) is reported once, identically in serial and parallel.

3 Test suite

The suite (`test/Allrun`) runs four tests, each with a summary log and a `results.json` read by this report:

Test	What it checks	Processors
`waveAccuracy`	accuracy against an exact wave	1 and 4
`specialCases`	interfaces and lines on faces, edges, processor and cyclic patches	1 and 8
`movingMesh`	line cells after adaptive refinement and mesh motion	1 and 4
`largeTiming`	cost in a large case	18

`largeTiming` runs on the last 18 cores, bound one process per core with local memory through an Open MPI rank file, while the other tests run on the remaining cores. The suite passes if the checks of `specialCases` and `movingMesh` pass; in the run reported here they passed and passed, respectively.

About `interfaceHeight`. `interfaceHeight` was added to OpenFOAM by the OpenFOAM Foundation in May 2017 (OpenFOAM-6) and has been in OpenFOAM.com since v2006, which added the optional `direction` entry. Its algorithm is the same in all OpenFOAM.com releases from v2006 to v2606: for each location it builds a `midPointAndFaceSet` along the mesh diagonal, with a new `meshSearch` at every write, integrates the interpolated volume fraction along it and sums the result over the processors. The only changes are the writing of the file headers (removed in v2012, restored on the master processor in v2406) and the documentation (v2512). The OpenFOAM Foundation corrected its parallel operation in May 2023 (commit `1d78434bee`, OpenFOAM-11) by skipping sampling segments between two faces; OpenFOAM.com does not have this correction. The tests use the v2512 version.

3.1 Accuracy: exact stream function wave

`streamFunFoam` sets the fields to an exact stream function wave (height 0.125 m, depth 0.4 m, period 2 s) in a periodic flume one wave length long, at every time step, and a coded function object writes the exact surface elevation at the lines. The flume has a coarse (38×8) and a

medium (95×20) mesh. Twelve vertical lines are placed in four cell columns, at the cell centre, a quarter cell off it and 2 % of a cell from a face (fig. 1). The elevations are compared over one period with 40 samples. Without a reconstruction the `plic` and `alpha` methods of `surfacePosition` are tested, against `interfaceHeight` with its default `cellPoint` and with `cell` interpolation.

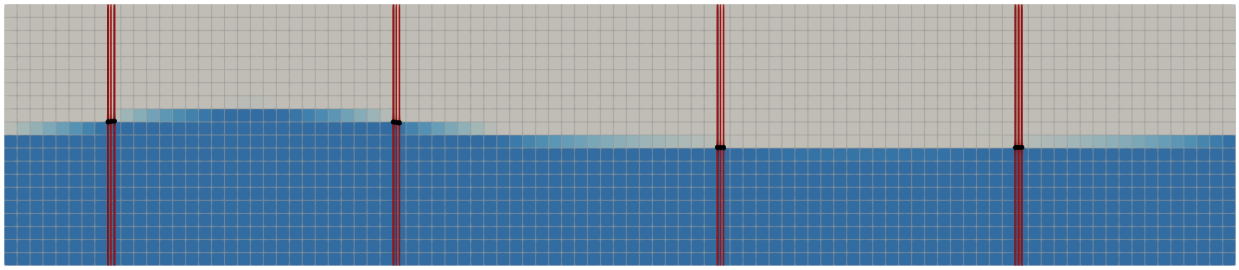


Figure 1: `waveAccuracy`, medium mesh at $t = 2\,\mathrm{s}$: volume fraction (white air, blue water), the 12 vertical lines (red, three per cell column) and the points found by `surfacePosition` (plic).

Table 1: Error of the interface elevation against the exact wave [mm]: rms per position of the line in the cell (in cell widths from the face), and mean and maximum magnitude over all lines and samples.

	rms, line position			all lines	
Method	0.50	0.25	0.02	mean	max
coarse, 38×8 cells					
`interfaceHeight`, cellPoint	0.499	0.562	0.511	0.000	1.631
`interfaceHeight`, cell	25.001	25.082	25.297	25.000	31.676
`surfacePosition`, plic	0.257	0.206	0.174	-0.001	0.741
`surfacePosition`, alpha	6.678	6.988	7.759	4.092	14.914
medium, 95×20 cells					
`interfaceHeight`, cellPoint	0.081	0.091	0.083	-0.000	0.267
`interfaceHeight`, cell	10.000	10.033	10.119	10.000	12.680
`surfacePosition`, plic	0.056	0.043	0.067	0.003	0.341
`surfacePosition`, alpha	2.159	2.305	2.661	-0.071	6.023

Table 1 shows that the plic method of `surfacePosition` has the smallest error on both meshes: 0.215 mm rms on the coarse mesh against 0.525 mm for `interfaceHeight` (cellPoint), and 0.056 mm against 0.085 mm on the medium mesh. `interfaceHeight` with cell interpolation is biased by a quarter of the cell height (25.0 mm and 10.0 mm). The alpha method, a linear interpolation of cell values, has errors of several millimetres. Fig. 2 shows the errors over the period at a cell centre and 2 % of a cell from a face. No method gave `nan`, and the serial and parallel results are identical.

3.2 Special cases

A closed $10 \times 10 \times 10$ cube, cyclic in x, decomposed into 8 subdomains, holds either a flat interface ($z = 0.537$ inside cells, $z = 0.5$ on internal and processor faces, $z = 0.3$ on internal faces) or a vertical slab of water ($x < 0.5$), whose interfaces lie on the cyclic and the processor faces. After one short `interIsoFoam` step, twelve vertical lines (fig. 3) test locations in a cell, on a face plane, on an edge, on a processor face and edge, on the cyclic patches, on a cyclic/processor edge and a cyclic/wall edge, far above and far below the mesh, and outside the domain. Six explicit lines test lines from below and from above, horizontal lines entering through the cyclic patches from outside

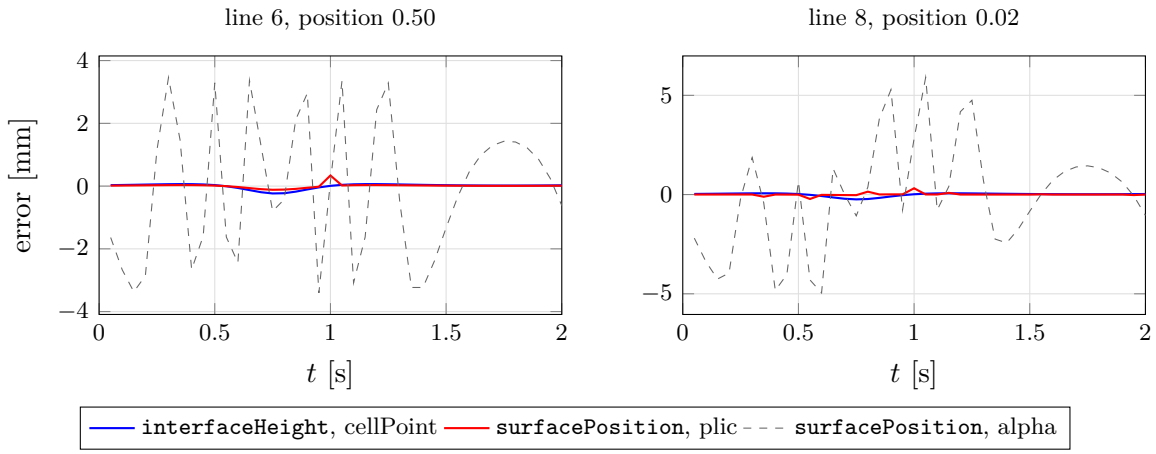
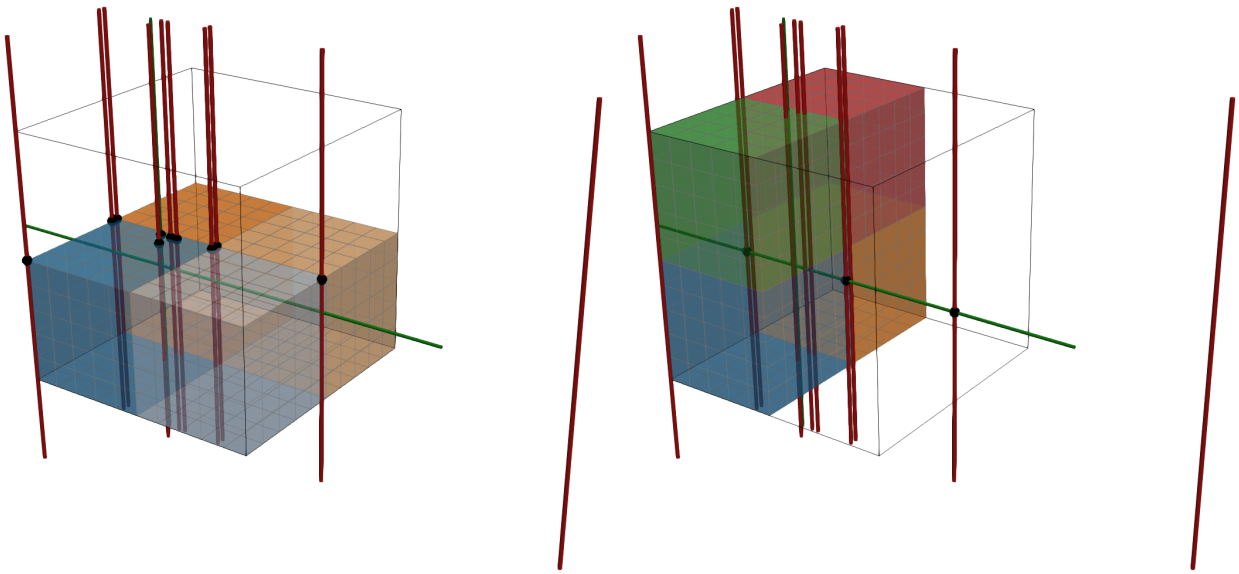


Figure 2: Error of the elevation over one wave period on the medium mesh, at a cell centre and 2 % of a cell width from a face.

the domain, and a line starting inside it. The results are compared with the exact positions; for the vertical slab the vertical lines must find no interface.



(a) Flat interface at $z = 0.5$

(b) Vertical slab of water

Figure 3: `specialCases` on 8 processors: water coloured by processor, vertical lines through the locations (red; the line on the right is outside the domain), explicit lines (green) and the points found by `surfacePosition` (plic).

All points are exact to round-off (table 2), in serial and on 8 processors, including the interfaces on processor and cyclic faces and the lines on faces, edges and patches. The alpha method is exact where the interface lies on a face and 7.6 mm off for $z = 0.537$, as expected for a linear interpolation. Table 3 shows `interfaceHeight` on the same lines for $z = 0.5$. It is exact in serial, but on 8 processors the lines on processor faces and edges count the water of both sides, and for locations far above or below the mesh, or outside it, it reports 0 instead of the interface or no interface.

3.3 Moving meshes

A 2D standing wave (amplitude 0.05 m) in a closed unit box with a 20×20 mesh, one cell thick in y, is run with `interIsoFoam` in two variants, in serial and on 4 processors:

Table 2: Largest error of the points found by `surfacePosition` against the exact positions [m], over the lines expected to find an interface; all other lines correctly gave `nan`. The tolerance for the geometric methods is 1×10^{-8} m; the alpha method is expected within half a cell.

		vertical lines			explicit lines	
Scenario	Proc.	reconstructed	plic	alpha	reconstructed	plic
flat, $z = 0.537$	1	3.3×10^{-16}	3.5×10^{-14}	0.0076	1.1×10^{-16}	3.5×10^{-14}
	8	3.3×10^{-16}	3.5×10^{-14}	0.0076	1.1×10^{-16}	3.5×10^{-14}
flat, $z = 0.5$	1	0	0	4.4×10^{-16}	4.4×10^{-16}	4.4×10^{-16}
	8	0	0	4.4×10^{-16}	4.4×10^{-16}	4.4×10^{-16}
flat, $z = 0.3$	1	1.7×10^{-16}	1.7×10^{-16}	2.8×10^{-16}	1.7×10^{-16}	1.7×10^{-16}
	8	1.7×10^{-16}	1.7×10^{-16}	2.8×10^{-16}	1.7×10^{-16}	1.7×10^{-16}
vertical slab	1	0	0	0	2.2×10^{-16}	2.2×10^{-16}
	8	0	0	0	2.2×10^{-16}	2.2×10^{-16}

Table 3: `interfaceHeight` (height h_B above the bottom [m]) on the special lines of the flat interface at $z = 0.5$.

Line	Location	Exact	Serial	8 processors
3	processor face	0.500	0.500	1.000
4	processor edge	0.500	0.500	1.975
7	cyclic/processor edge	0.500	0.500	0.900
8	far above	0.500	0.000	0.000
9	far below	0.500	0.000	0.000
10	outside the domain	–	0.000	0.000

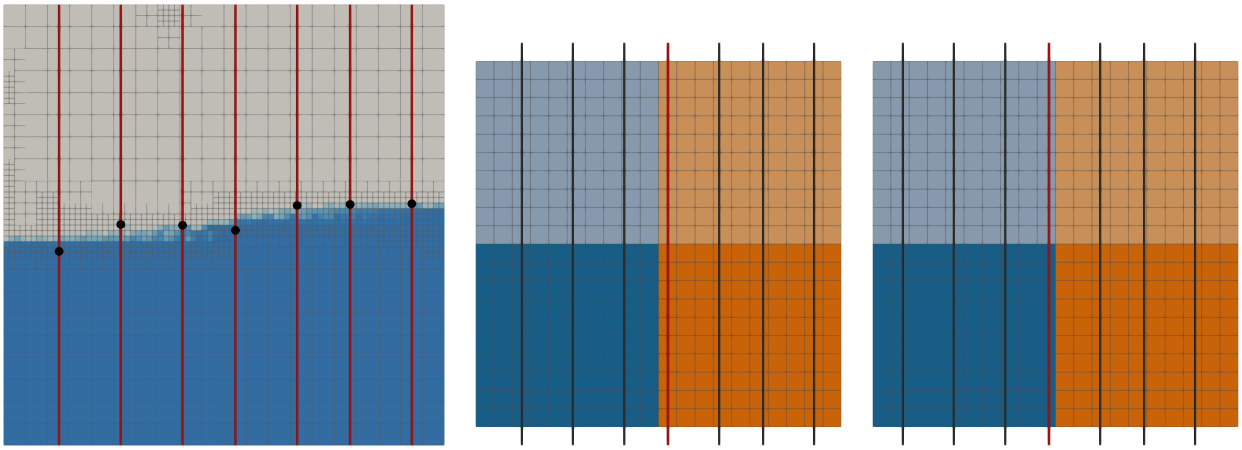
amr adaptive refinement of the interface cells to two levels at every time step with `dynamicRefineFvMesh`, for 400 time steps. The refinement also splits the cells in y, so that the seven lines, which lie in the plane $y = 0$, run in the internal faces of the refined cells (fig. 4a).

motion a solid body oscillation of the whole mesh in x, $0.05\,\text{m}\,\sin(5.3\,t)$, for 0.6 s, with the lines fixed in space, so that the line cells change at every time step.

Table 4: `movingMesh`: line cells after all mesh changes compared with all cells (`checkLineCells`), outputs without an interface, and rms difference of the elevation from `interfaceHeight` [mm].

				line cells				rms vs. `interfaceHeight`	
Case	Proc.	Steps	Changes	found	missing	extra	`nan`	reconstr.	plic
amr	1	400	177	48730	0	0	0	41.20	46.58
amr	4	400	396	109857	0	0	0	33.58	33.92
motion	1	51	50	7000	0	0	0	1.84	0.61
motion	4	51	50	7000	0	0	0	1.84	0.60

In all runs the flood fill finds exactly the cells crossing the lines (table 4): 177 and 396 mesh changes in the serial and parallel amr runs, 50 in the motion runs. In the parallel motion run, line 3 moves from the subdomains right of a processor boundary to those left of it between $t = 0.1$ and 0.1125 s (fig. 4); the flood fill follows it across the processor faces and finds all 7000 line cells, as in serial.



(a) amr, $t = 0.0818\,\mathrm{s}$ (b) motion, $t = 0$ (c) motion, $t = 0.2\,\mathrm{s}$

Figure 4: `movingMesh`. (a) Refined mesh, volume fraction, lines and the points found by `surfacePosition` (reconstructed). (b, c) Mesh coloured by processor on 4 processors with line 3 (red): the mesh moves right, and line 3 passes from the right to the left subdomains.

The differences from `interfaceHeight` in the amr runs (table 4, maximum 461 mm) are not errors of either function object: the refinement produces spurious velocities that limit the time step to about 0.000 21 s and throw up droplets, so the first interface from above and the integrated water height differ. For the same reason serial and parallel amr runs diverge (`interfaceHeight` differs by up to 10.8 mm). In the motion runs, serial and parallel flows differ slightly, by up to 0.13 mm in `interfaceHeight` and 0.67 mm in `surfacePosition`.

3.4 Cost in a large case

A 3D standing wave in a closed unit cube with 1 000 000 cells is run with `interIsoFoam` on 18 processors (scotch decomposition), with 50 vertical lines placed off the cell centres (fig. 5) and `surfacePosition` (reconstructed and plic) and `interfaceHeight` writing at every time step. OpenFOAM's profiling gives the time of each function object and of the flow solution; it is written with the fields after 5 start-up steps and at the end, and the difference gives the time of 10 steps.

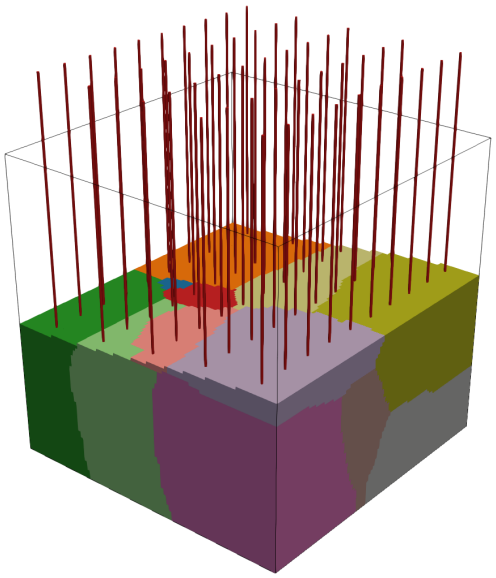


Figure 5: `largeTiming`: water region coloured by the 18 processor subdomains, and the 50 lines.

Table 5: Time per step (mean over the processors, each including the waiting for the others), per line, the range over the processors, and the start-up time of the first steps beyond the steady cost (the line search), in ms.

	per step	per line	range	start-up
flow solution (`interIsoFoam`)	752.1		1.20	
`surfacePosition`, reconstructed	2.4	0.048	1.34	1130
`surfacePosition`, plic	2.9	0.059	0.04	953
`interfaceHeight`	1278.4	25.569	0.15	

`surfacePosition` costs 2.4 ms per step (2.9 ms with plic), 0.3 % of the flow solution, while `interfaceHeight` costs 1278 ms, 170 % of the flow solution and 538 times as much as `surfacePosition` (table 5). The initial line search of `surfacePosition` takes about 1130 ms once. The elevations of `surfacePosition` and `interfaceHeight` agree to 0.108 mm (reconstructed, rms 0.043 mm) and 0.079 mm (plic).

4 Defects found by the tests

Two defects of `surfacePosition` were found by the moving mesh tests and fixed:

1. *Round-off in the test for faces parallel to the line* (`crossesLine`). A face was taken as parallel to the line only if $|a| \le 10^{-300}|\mathbf{S}_f|$. For the lines in the face planes $y = 0$ of the refined cells, round-off gave $|a|/|\mathbf{S}_f| \approx 10^{-17}$, so b/a became an arbitrary distance that could clip a crossed cell away and disconnect the flood fill. In a separate run of the serial amr case with the old test, all of its 177 mesh changes missed line cells; with eq. (1) none does.

2. *Lines lost across processor boundaries.* The flood fill followed internal faces only. When the motion moved line 3 into the neighbouring subdomains, neither side found any cell, and `surfacePosition` wrote `nan` for that line for the rest of the run. The flood fill now continues across processor faces.

The checks of the suite were tightened accordingly: `nan` outputs are counted and fail the moving mesh test, and the check scripts' exit codes are passed on to `Allrun`.

The tests also show that `interfaceHeight` (v2512) is wrong on processor faces and for locations outside the vertical extent of the mesh (table 3). The first may be what the OpenFOAM Foundation's parallel correction addresses (section 3); this was not tested.

5 Reproducing this report

With OpenFOAM v2512 sourced, in the repository of `surfacePosition`:

```
./Allwmake            # libsurfacePosition
cd test
./Allrun              # the test suite, results.json per test
cd report
./Allrun              # makeData.py, makeFigures.py (pvbatch), latexmk
```

`makeData.py` writes the numbers and tables in `generated/` from the `results.json` files and the `postProcessing` data of the cases; `makeFigures.py` renders the figures from the case data with ParaView. This report was generated from the run of 2026-09-17 07:08 (source and tests at revision `1ad71fe`).